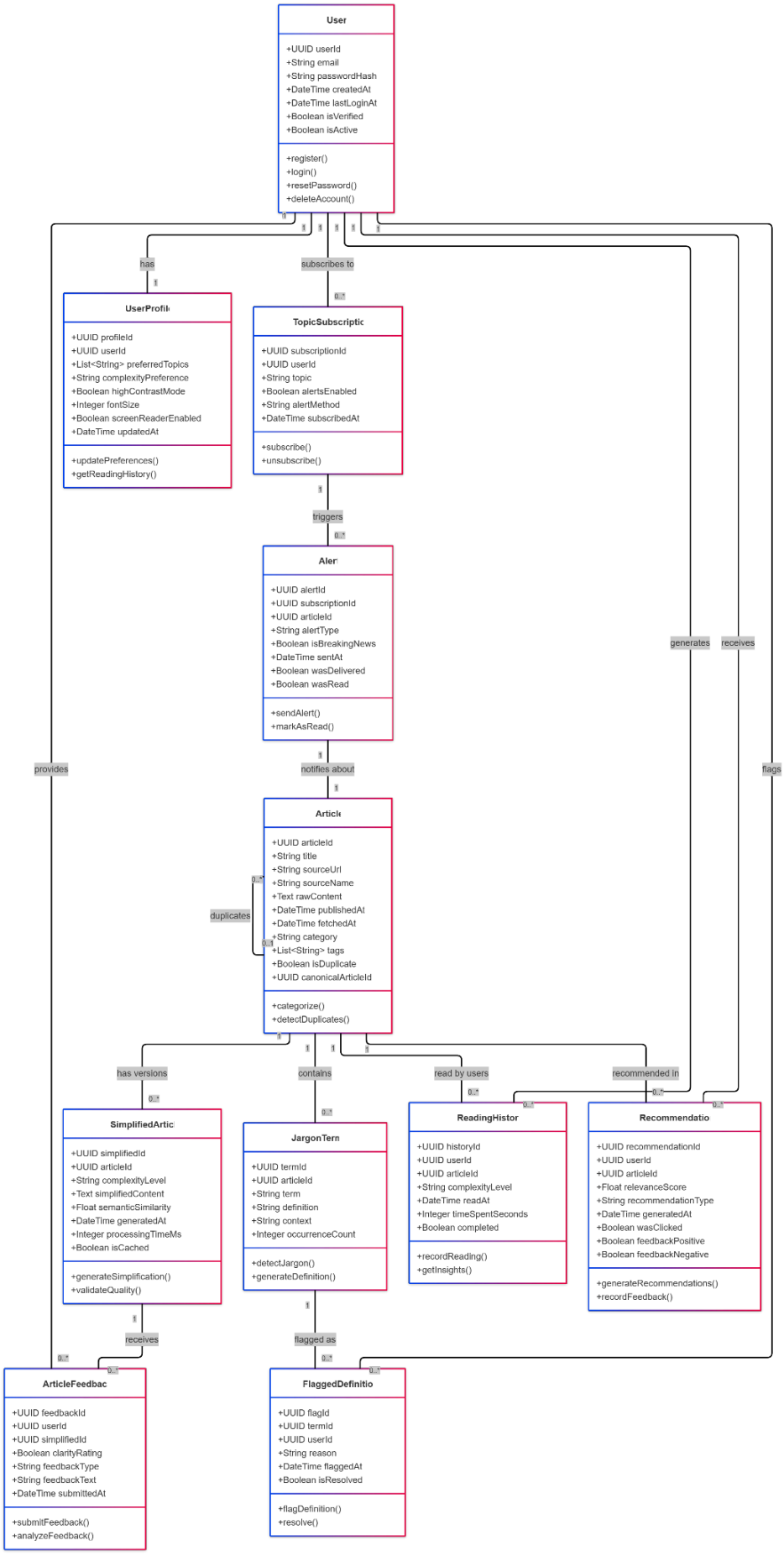
News For U Domain Model (UML Class Diagram)



2.1 Domain Model Explanation

1. **User**: Represents authenticated platform users
    - Supports registration, login, password reset, and account deletion (Epic 1)
    - Links to profile, reading history, and feedback
2. **UserProfile**: Stores user preferences and accessibility settings
    - Preferred topics, complexity level, font size, contrast mode
    - Screen reader compatibility flag
    - Drives personalization logic
3. **Article**: Raw news content from external sources
    - Canonical entity for all content
    - Supports deduplication through self-referential relationship
    - Tagged with category and metadata
4. **SimplifiedArticle**: AI-generated versions at different complexity levels
    - One-to-many relationship with Article (beginner, intermediate, expert)
    - Semantic similarity score tracks quality
    - Cached flag optimizes performance
5. **JargonTerm**: Technical terms extracted from articles
    - Contextual definitions for tooltips (Story 3.4)
    - Occurrence count tracks term frequency
6. **ReadingHistory**: User engagement tracking
    - Records complexity level preference per article
    - Time spent enables recommendation tuning
    - Completion status indicates engagement quality
7. **Recommendation**: Personalized article suggestions
    - Relevance score from ML model
    - Feedback (useful/not useful) trains recommendation engine (Story 4.3)
8. **TopicSubscription**: User-selected news topics for alerts
    - Supports email and push notification preferences (Epic 7)
9. **Alert**: Breaking news and topic-based notifications
    - Delivery tracking prevents duplicates
    - Read status enables engagement metrics
10. **ArticleFeedback**: User ratings on simplified content
    - Yes/No clarity ratings (Story 6.1)
    - Trains AI models for continuous improvement
11. **FlaggedDefinition**: User-reported unclear definitions
    - Admin review workflow (Story 6.2)
    - Resolution tracking for quality control

Key Relationships:

- Users have one profile, multiple reading histories, recommendations, and subscriptions
- Articles have multiple simplified versions and jargon terms
- Recommendations link users to articles with feedback loop
- Subscriptions trigger alerts for relevant articles

---

3. Conceptual Data Model

3.1 Entities and Attributes

User

- userId (UUID, PK)
- email (String, Unique, Not Null)
- passwordHash (String, Not Null)
- createdAt (Timestamp, Not Null)
- lastLoginAt (Timestamp, Nullable)
- isVerified (Boolean, Default: False)
- isActive (Boolean, Default: True)

UserProfile

- profileId (UUID, PK)
- userId (UUID, FK → User, Unique, Not Null)
- preferredTopics (Array[String], Default: [])
- complexityPreference (Enum: beginner|intermediate|expert, Default: intermediate)
- highContrastMode (Boolean, Default: False)
- fontSize (Integer, Range: 12-24, Default: 16)
- screenReaderEnabled (Boolean, Default: False)
- updatedAt (Timestamp, Not Null)

Article

- articleId (UUID, PK)
- title (String, Not Null)
- sourceUrl (String, Unique, Not Null)
- sourceName (String, Not Null)
- rawContent (Text, Not Null)
- publishedAt (Timestamp, Not Null)
- fetchedAt (Timestamp, Not Null)
- category (Enum: politics|technology|health|finance|sports|other, Not Null)
- tags (Array[String], Default: [])
- isDuplicate (Boolean, Default: False)
- canonicalArticleId (UUID, FK → Article, Nullable)

SimplifiedArticle

- simplifiedId (UUID, PK)
- articleId (UUID, FK → Article, Not Null)
- complexityLevel (Enum: beginner|intermediate|expert, Not Null)
- simplifiedContent (Text, Not Null)
- semanticSimilarity (Float, Range: 0.0-1.0, Nullable)

- generatedAt (Timestamp, Not Null)
- processingTimeMs (Integer, Not Null)
- isCached (Boolean, Default: True)
- Unique Constraint: (articleId, complexityLevel)

JargonTerm

- termId (UUID, PK)
- articleId (UUID, FK → Article, Not Null)
- term (String, Not Null)
- definition (Text, Not Null)
- context (String, Nullable)
- occurrenceCount (Integer, Default: 1)

ReadingHistory

- historyId (UUID, PK)
- userId (UUID, FK → User, Not Null)
- articleId (UUID, FK → Article, Not Null)
- complexityLevel (Enum: beginner|intermediate|expert, Not Null)
- readAt (Timestamp, Not Null)
- timeSpentSeconds (Integer, Nullable)
- completed (Boolean, Default: False)
- Unique Constraint: (userId, articleId, readAt)

Recommendation

- recommendationId (UUID, PK)
- userId (UUID, FK → User, Not Null)
- articleId (UUID, FK → Article, Not Null)
- relevanceScore (Float, Range: 0.0-1.0, Not Null)
- recommendationType (Enum: collaborative|content_based|hybrid, Not Null)
- generatedAt (Timestamp, Not Null)
- wasClicked (Boolean, Default: False)
- feedbackPositive (Boolean, Nullable)
- feedbackNegative (Boolean, Nullable)

TopicSubscription

- subscriptionId (UUID, PK)
- userId (UUID, FK → User, Not Null)
- topic (String, Not Null)
- alertsEnabled (Boolean, Default: True)
- alertMethod (Enum: email|push|none, Default: push)
- subscribedAt (Timestamp, Not Null)
- Unique Constraint: (userId, topic)

Alert

- alertId (UUID, PK)
- subscriptionId (UUID, FK → TopicSubscription, Not Null)
- articleId (UUID, FK → Article, Not Null)
- alertType (Enum: breaking_news|topic_update, Not Null)
- isBreakingNews (Boolean, Default: False)
- sentAt (Timestamp, Not Null)
- wasDelivered (Boolean, Default: False)
- wasRead (Boolean, Default: False)

ArticleFeedback

- feedbackId (UUID, PK)
- userId (UUID, FK → User, Not Null)
- simplifiedId (UUID, FK → SimplifiedArticle, Not Null)
- clarityRating (Boolean, Not Null)
- feedbackType (Enum: clarity|accuracy|relevance, Default: clarity)
- feedbackText (Text, Nullable)
- submittedAt (Timestamp, Not Null)

FlaggedDefinition

- flagId (UUID, PK)
- termId (UUID, FK → JargonTerm, Not Null)
- userId (UUID, FK → User, Not Null)
- reason (String, Nullable)
- flaggedAt (Timestamp, Not Null)
- isResolved (Boolean, Default: False)

3.2 Relationships

1. User ↔ UserProfile (1:1, mandatory)
2. User ↔ ReadingHistory (1:N)
3. User ↔ Recommendation (1:N)
4. User ↔ TopicSubscription (1:N)
5. User ↔ ArticleFeedback (1:N)
6. User ↔ FlaggedDefinition (1:N)
7. Article ↔ SimplifiedArticle (1:N)
8. Article ↔ JargonTerm (1:N)
9. Article ↔ Article (N:1, self-referential for duplicates)
10. Article ↔ ReadingHistory (1:N)
11. Article ↔ Recommendation (1:N)
12. SimplifiedArticle ↔ ArticleFeedback (1:N)
13. JargonTerm ↔ FlaggedDefinition (1:N)
14. TopicSubscription ↔ Alert (1:N)
15. Alert ↔ Article (N:1)

3.3 Constraints

- **Referential Integrity**: All foreign keys enforce cascade delete (except Article.canonicalArticleId, which sets NULL on delete)
- **Uniqueness**: Emails, source URLs, and (userId, topic) pairs must be unique
- **Check Constraints**: Semantic similarity ∈ [0.0, 1.0]; fontSize ∈ [12, 24]; relevanceScore ∈ [0.0, 1.0]
- **Mutual Exclusivity**: feedbackPositive and feedbackNegative cannot both be true

---

4. Logical Data Model

4.1 Table Schemas

users

Column	Type	Constraints
user_id	UUID	PRIMARY KEY
email	VARCHAR(255)	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	NOT NULL
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()
last_login_at	TIMESTAMP	NULL
is_verified	BOOLEAN	NOT NULL, DEFAULT FALSE
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE

Indexes: `idx_users_email` (B-tree), `idx_users_active` (B-tree on is_active)

---

user_profiles

Column	Type	Constraints
profile_id	UUID	PRIMARY KEY
user_id	UUID	FOREIGN KEY → users(user_id), UNIQUE, NOT NULL
preferred_topics	TEXT[]	DEFAULT '{}'
complexity_preference	VARCHAR(20)	NOT NULL, DEFAULT 'intermediate', CHECK IN ('beginner','intermediate','expert')
high_contrast_mode	BOOLEAN	NOT NULL, DEFAULT FALSE

font_size	INTEGER	NOT NULL, DEFAULT 16, CHECK (font_size BETWEEN 12 AND 24)
screen_reader_enabled	BOOLEAN	NOT NULL, DEFAULT FALSE
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()

Indexes: `idx_user_profiles_user_id` (B-tree)

---

articles

Column	Type	Constraints
article_id	UUID	PRIMARY KEY
title	VARCHAR(500)	NOT NULL
source_url	TEXT	UNIQUE, NOT NULL
source_name	VARCHAR(255)	NOT NULL
raw_content	TEXT	NOT NULL
published_at	TIMESTAMP	NOT NULL
fetched_at	TIMESTAMP	NOT NULL, DEFAULT NOW()
category	VARCHAR(50)	NOT NULL, CHECK IN ('politics','technology','health','finance','sports','other')
tags	TEXT[]	DEFAULT '{}'
is_duplicate	BOOLEAN	NOT NULL, DEFAULT FALSE
canonical_article_id	UUID	FOREIGN KEY → articles(article_id), NULL, ON DELETE SET NULL

Indexes:

- `idx_articles_published_at` (B-tree, DESC for recent articles)
- `idx_articles_category` (B-tree)
- `idx_articles_source_url` (Hash)
- `idx_articles_title_fulltext` (GIN for full-text search)

simplified_articles

Column	Type	Constraints
simplified_id	UUID	PRIMARY KEY
article_id	UUID	FOREIGN KEY → articles(article_id), NOT NULL, ON DELETE CASCADE
complexity_level	VARCHAR(20)	NOT NULL, CHECK IN ('beginner','intermediate','expert')
simplified_content	TEXT	NOT NULL
semantic_similarity	FLOAT	NULL, CHECK (semantic_similarity BETWEEN 0.0 AND 1.0)
generated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()
processing_time_ms	INTEGER	NOT NULL
is_cached	BOOLEAN	NOT NULL, DEFAULT TRUE
UNIQUE (article_id, complexity_level)		

Indexes: `idx_simplified_articles_article_id` (B-tree), `idx_simplified_articles_generated_at` (B-tree)

---

jargon_terms

Column	Type	Constraints
term_id	UUID	PRIMARY KEY
article_id	UUID	FOREIGN KEY → articles(article_id), NOT NULL, ON DELETE CASCADE
term	VARCHAR(255)	NOT NULL
definition	TEXT	NOT NULL

context	TEXT	NULL
occurrence_count	INTEGER	NOT NULL, DEFAULT 1

Indexes: `idx_jargon_terms_article_id` (B-tree), `idx_jargon_terms_term` (B-tree)

---

reading_history

Column	Type	Constraints
history_id	UUID	PRIMARY KEY
user_id	UUID	FOREIGN KEY → users(user_id), NOT NULL, ON DELETE CASCADE
article_id	UUID	FOREIGN KEY → articles(article_id), NOT NULL, ON DELETE CASCADE
complexity_level	VARCHAR(20)	NOT NULL, CHECK IN ('beginner','intermediate','expert')
read_at	TIMESTAMP	NOT NULL, DEFAULT NOW()
time_spent_seconds	INTEGER	NULL
completed	BOOLEAN	NOT NULL, DEFAULT FALSE

Indexes:

- `idx_reading_history_user_id` (B-tree)
- `idx_reading_history_article_id` (B-tree)
- `idx_reading_history_read_at` (B-tree, DESC)

---

recommendations

Column	Type	Constraints
recommendation_id	UUID	PRIMARY KEY

user_id	UUID	FOREIGN KEY → users(user_id), NOT NULL, ON DELETE CASCADE
article_id	UUID	FOREIGN KEY → articles(article_id), NOT NULL, ON DELETE CASCADE
relevance_score	FLOAT	NOT NULL, CHECK (relevance_score BETWEEN 0.0 AND 1.0)
recommendation_type	VARCHAR(20)	NOT NULL, CHECK IN ('collaborative','content_based','hybrid')
generated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()
was_clicked	BOOLEAN	NOT NULL, DEFAULT FALSE
feedback_positive	BOOLEAN	NULL
feedback_negative	BOOLEAN	NULL, CHECK (NOT (feedback_positive AND feedback_negative))

Indexes:

- `idx_recommendations_user_id` (B-tree)
- `idx_recommendations_generated_at` (B-tree, DESC)

---

topic_subscriptions

Column	Type	Constraints
subscription_id	UUID	PRIMARY KEY
user_id	UUID	FOREIGN KEY → users(user_id), NOT NULL, ON DELETE CASCADE
topic	VARCHAR(100)	NOT NULL
alerts_enabled	BOOLEAN	NOT NULL, DEFAULT TRUE
alert_method	VARCHAR(20)	NOT NULL, DEFAULT 'push', CHECK IN ('email','push','none')
subscribed_at	TIMESTAMP	NOT NULL, DEFAULT NOW()

UNIQUE (user_id, topic)		

Indexes: `idx_topic_subscriptions_user_id` (B-tree)

---

alerts

Column	Type	Constraints
alert_id	UUID	PRIMARY KEY
subscription_id	UUID	FOREIGN KEY → topic_subscriptions(subscription_id), NOT NULL, ON DELETE CASCADE
article_id	UUID	FOREIGN KEY → articles(article_id), NOT NULL, ON DELETE CASCADE
alert_type	VARCHAR(20)	NOT NULL, CHECK IN ('breaking_news','topic_update')
is_breaking_news	BOOLEAN	NOT NULL, DEFAULT FALSE
sent_at	TIMESTAMP	NOT NULL,DEFAULT NOW()
was_delivered	BOOLEAN	NOT NULL, DEFAULT FALSE
was_read	BOOLEAN	NOT NULL, DEFAULT FALSE

Indexes:

- `idx_alerts_subscription_id` (B-tree)
- `idx_alerts_sent_at` (B-tree, DESC)
- `idx_alerts_article_id` (B-tree)

article_feedback

Column	Type	Constraints
feedback_id	UUID	PRIMARY KEY
user_id	UUID	FOREIGN KEY → users(user_id), NOT NULL, ON DELETE CASCADE
simplified_id	UUID	FOREIGN KEY → simplified_articles(simplified_id), NOT NULL, ON DELETE CASCADE
clarity_rating	BOOLEAN	NOT NULL
feedback_type	VARCHAR(20)	NOT NULL, DEFAULT 'clarity', CHECK IN ('clarity','accuracy','relevance')
feedback_text	TEXT	NULL
submitted_at	TIMESTAMP	NOT NULL, DEFAULT NOW()

Indexes:

- `idx_article_feedback_user_id` (B-tree)
- `idx_article_feedback_simplified_id` (B-tree)
- `idx_article_feedback_submitted_at` (B-tree, DESC)

---

flagged_definitions

Column	Type	Constraints
flag_id	UUID	PRIMARY KEY
term_id	UUID	FOREIGN KEY → jargon_terms(term_id), NOT NULL, ON DELETE CASCADE
user_id	UUID	FOREIGN KEY → users(user_id), NOT NULL, ON DELETE CASCADE
reason	TEXT	NULL
flagged_at	TIMESTAMP	NOT NULL, DEFAULT NOW()
is_resolved	BOOLEAN	NOT NULL, DEFAULT FALSE

Indexes:

- `idx_flagged_definitions_term_id` (B-tree)
- `idx_flagged_definitions_is_resolved` (B-tree on is_resolved WHERE is_resolved = FALSE)

4.2 Derived Attributes

The following attributes are computed at query time and not stored:

1. **User.totalArticlesRead**: COUNT of ReadingHistory entries
2. **User.preferredComplexityLevel**: MODE of ReadingHistory.complexityLevel
3. **Article.averageReadTime**: AVG(ReadingHistory.timeSpentSeconds)
4. **Article.popularityScore**: Weighted combination of read count and recent views
5. **SimplifiedArticle.avgClarityRating**: AVG(ArticleFeedback.clarityRating) as percentage
6. **Recommendation.clickThroughRate**: (wasClicked count / total recommendations) per user

5. Data Quality & Integrity Rules

5.1 Uniqueness Constraints

Entity	Unique Constraint	Business Rule
User	email	Prevent duplicate accounts; case-insensitive comparison
Article	source_url	One canonical article per URL; duplicates reference via canonical_article_id
SimplifiedArticle	(article_id, complexity_level)	Only one simplified version per level per article
TopicSubscription	(user_id, topic)	Users cannot subscribe to same topic twice

5.2 Referential Integrity

Cascade Delete Rules:

- Deleting a **User** cascades to: UserProfile, ReadingHistory, Recommendations, TopicSubscriptions, ArticleFeedback, FlaggedDefinitions
- Deleting an **Article** cascades to: SimplifiedArticles, JargonTerms, ReadingHistory, Recommendations, Alerts
- Deleting a **TopicSubscription** cascades to: Alerts

Set NULL Rules:

- Deleting an **Article** with duplicates sets `canonical_article_id` to NULL in dependent articles (orphaned duplicates become standalone)

Restrict Delete:

- Cannot delete **JargonTerms** with unresolved FlaggedDefinitions (admin must resolve first)

5.3 Completeness Rules

Mandatory Fields (NOT NULL):

- All primary keys and foreign keys (except optional relationships like canonical_article_id)
- User email, password_hash (authentication requirement)
- Article title, content, source (content integrity)
- SimplifiedArticle content, complexity_level (functional requirement)
- Timestamps: created_at, fetched_at, generated_at (audit trail)

Optional Fields (Nullable):

- User.last_login_at (may be first-time user)
- ReadingHistory.time_spent_seconds (may not capture complete session)
- ArticleFeedback.feedback_text (optional elaboration)
- SimplifiedArticle.semantic_similarity (LLM may not return this metric)

5.4 Domain Integrity

Enumeration Constraints:

- `complexity_preference`, `complexity_level` ∈ {beginner, intermediate, expert}
- `category` ∈ {politics, technology, health, finance, sports, other}
- `alert_method` ∈ {email, push, none}
- `alert_type` ∈ {breaking_news, topic_update}
- `recommendation_type` ∈ {collaborative, content_based, hybrid}

Range Constraints:

- `font_size` ∈ [12, 24] (accessibility requirement)
- `semantic_similarity` ∈ [0.0, 1.0] (normalized score)
- `relevance_score` ∈ [0.0, 1.0] (normalized score)
- `processing_time_ms` > 0 (positive duration)

Logical Constraints:

- `feedback_positive` AND `feedback_negative` cannot both be TRUE (mutual exclusivity)
- `published_at` ≤ `fetched_at` (cannot fetch before publication)
- `generated_at` ≥ `article.fetched_at` (simplification happens after fetching)

5.5 Data Validation Rules

Email Validation:

sql

```
CHECK (email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$')
```

URL Validation:

sql

```
CHECK (source_url ~* '^https?://')
```

Password Policy (Application Layer):

- Minimum 8 characters
- At least one uppercase, one lowercase, one digit, one special character

- Not in common password breach lists (checked via API)

Content Length Validation:

- `article.raw_content`: 100-50,000 characters (filter out snippets and excessively long content)
- `simplified_article.simplified_content`: 50-40,000 characters (ensure meaningful simplification)
- `jargon_term.definition`: 10-500 characters (concise but complete)

5.6 Temporal Integrity

Timestamp Consistency:

- `article.published_at` must be within past 30 days for trending calculations
- `simplified_articles.generated_at` must be after parent article's `fetched_at`
- `reading_history.read_at` cannot be in the future
- Alert `sent_at` must be within 5 minutes of article detection for breaking news

Session Timeout:

- Redis sessions expire after 30 minutes of inactivity (per success criteria)
- Cached simplified articles expire after 24 hours
- Rate limit counters reset hourly

5.7 Business Logic Integrity

Deduplication Invariant:

- If `article.is_duplicate = TRUE`, then `canonical_article_id` must NOT be NULL
- Canonical articles must have `is_duplicate = FALSE`
- No circular references in canonical_article_id chain

Recommendation Constraints:

- Users should not receive recommendations for articles already in ReadingHistory
- Implemented via application logic: `WHERE article_id NOT IN (SELECT article_id FROM reading_history WHERE user_id = ?)`

Alert Deduplication:

- Do not send multiple alerts for same article to same subscription within 24 hours
- Enforced via unique index: `CREATE UNIQUE INDEX idx_unique_alert ON alerts(subscription_id, article_id, DATE(sent_at))`

6. Accessibility Compliance Details

6.1 WCAG 2.1 AA Requirements

Perceivable:

- Alt text for all images (Article thumbnails)
- Color contrast ratio ≥ 4.5:1 for normal text, ≥ 3:1 for large text
- Text resize up to 200% without loss of functionality (font_size setting)
- High contrast mode toggle in UserProfile

Operable:

- Full keyboard navigation (React Aria components)
- No keyboard traps; logical tab order
- Skip navigation links for screen readers
- Adjustable time limits for reading (no auto-refresh)

Understandable:

- Clear error messages for forms (registration, login)
- Consistent navigation across pages
- Jargon definitions on hover/focus (Story 3.4)

Robust:

- Semantic HTML5 (article, nav, main, aside tags)
- ARIA labels and roles (aria-label, role="navigation")
- Screen reader compatibility tested with NVDA, JAWS, VoiceOver

6.2 Implementation in Architecture

Frontend (React/Next.js):

- React
- Aria components provide built-in accessibility patterns
- Tailwind CSS utility classes include focus-visible states
- Dynamic font size adjustment via CSS custom properties tied to UserProfile.fontSize
- High contrast mode applies alternate stylesheet based on UserProfile.highContrastMode

Backend Support:

- UserProfile table stores accessibility preferences
- API responses include semantic metadata (article structure, reading level indicators)
- Alternative text generation for article images via image recognition API (future enhancement)

Testing Strategy:

- Automated accessibility testing: axe-core, Lighthouse CI in GitHub Actions
- Manual testing with screen readers: NVDA (Windows), JAWS (Windows), VoiceOver (macOS/iOS)
- Keyboard-only navigation testing before each sprint release
- Color contrast verification tools: WebAIM Contrast Checker

7. Security Considerations

7.1 Authentication & Authorization

JWT Token Structure:

```
{
  "user_id": "uuid",
  "email": "user@example.com",
  "roles": ["user"],
  "iat": 1698765432,
  "exp": 1698769032
}
```

Token Lifecycle:

- Access token: 15-minute expiry (short-lived for security)
- Refresh token: 7-day expiry (stored in HttpOnly cookie)
- Token refresh endpoint: `/api/v1/auth/refresh`

Authorization Levels:

- **User**: Read articles, provide feedback, manage own profile
- **Admin**: Review flagged definitions, access analytics dashboard, manage content sources
- **System**: Internal service-to-service authentication (API keys in environment variables)

7.2 Data Security

Encryption:

- **In Transit**: TLS 1.3 for all API communications
- **At Rest**:
  - PostgreSQL: Transparent Data Encryption (TDE)
  - S3: Server-side encryption with AWS KMS
  - Redis: Encryption enabled for data persistence

Password Security:

- Hashing: bcrypt with work factor 12 (balances security and performance)
- Salt: Unique per-user salt automatically generated
- Password reset tokens: Single-use, 30-minute expiry, stored as hashed values

Sensitive Data Handling:

- PII (email, reading history) stored in compliance with GDPR
- API keys and secrets stored in AWS Secrets Manager
- No sensitive data in application logs (redact passwords, tokens)

7.3 Input Validation & Sanitization

Frontend Validation:

- React Hook Form with Zod schema validation
- Real-time feedback on invalid inputs

Backend Validation:

- Express Validator middleware for all API endpoints
- SQL injection prevention: Parameterized queries (prepared statements)
- XSS prevention: Content Security Policy headers, HTML sanitization with DOMPurify

Example Validation Schema:

```
const articleSearchSchema = {
  query: {
    type: 'string',
    minLength: 2,
    maxLength: 200,
    pattern: '^[a-zA-Z0-9\\s\\-]+$'
  },
  category: {
    type: 'string',
    enum: ['politics', 'technology', 'health', 'finance', 'sports', 'other'],
    optional: true
  }
};
```